



Desarrollo Backend II

(PBY2202)

Nombre estudiante: Francisco Henríquez – Raúl Zúñiga	
Asignatura: Desarrollo Backend II	Carrera: Desarrollo de Aplicaciones
Profesor: Jorge Carmona	Fecha: 19-07-2026

Contenido

Descripción de la actividad	3
Definición de actividades del proyecto	4
Configuración del frameworks de seguridad	8
Detalle de las pruebas unitarias	14
Mejoras aplicadas	20
Documentación técnica de la API utilizando estándares OpenAPI y Specification (OAS) y HEATEOAS	22

Descripción de la actividad

Durante esta novena semana se llevó a cabo la Evaluación Final Transversal (EFT) de la asignatura, la cual consiste en el desarrollo de una aplicación backend grupal, en la que se debió aplicar los conocimientos adquiridos a través del caso planteado de Minimarket Plus.

La presente actividad se compone de dos partes:

- A- En la primera, se implementó una solución técnica que integra microservicios, frameworks de seguridad, pruebas unitarias y documentación bajo estándares como OpenAPI Specification (OAS) y HATEOAS, donde se evidenció la configuración, resultados y mejoras aplicadas;
- B- En la segunda, se presentó el funcionamiento de la aplicación mediante un video, explicando detalladamente cada uno de los procesos desarrollados, con énfasis en su ejecución, funcionalidad y las conclusiones alcanzadas a lo largo de la experiencia de aprendizaje.

Definición de actividades del proyecto

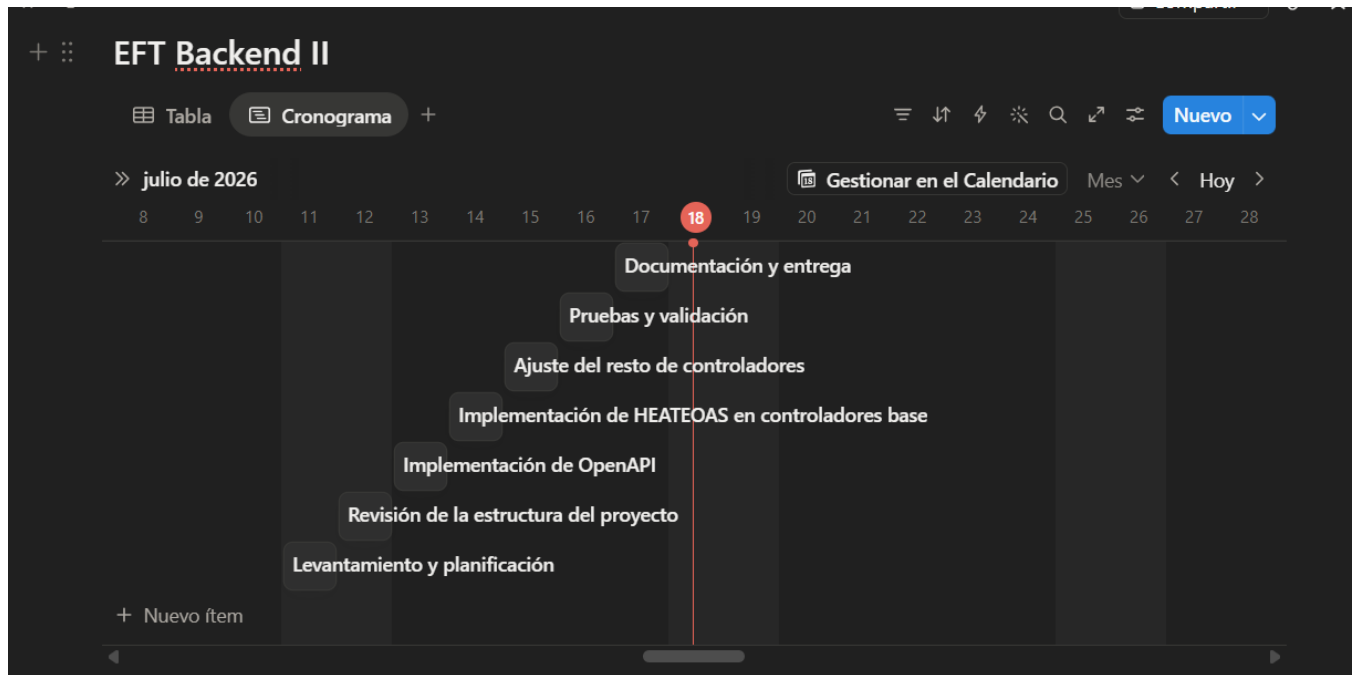
El proyecto lo comenzamos a trabajar desde el momento que se habilitó en el portal del AVA, La definición de las actividades fueron plasmadas utilizando la Herramienta Notion, la cual nos permitió definir las actividades para cada uno de los participantes del grupo y además poder visualizar inmediatamente el cronograma de las actividades.

Imagen de las actividades seleccionadas en Notion para cada integrante del grupo



Nombre	Fecha	Texto
Levantamiento y planificación	11 de julio de 2026	- Revisar los requisitos de la actividad. - Identificar entidades, controladores y servicios existentes. - Definir qué partes se documentarán con OpenAPI y cuáles se reforzarán con HATEOAS. - Distribuir responsabilidades entre las dos personas.
Revisión de la estructura del proyecto	12 de julio de 2026	- Analizar la arquitectura del backend. - Verificar las dependencias en <code>pom.xml</code>. - Revisar la configuración de seguridad, Swagger y las rutas protegidas. - Identificar controladores que aún devolvían respuestas simples.
Implementación de OpenAPI	13 de julio de 2026	- Agregar o completar las anotaciones <code>@Tag</code>, <code>@Operation</code> y <code>@ApiResponse</code>. - Documentar los endpoints principales. - Verificar que Swagger UI muestre correctamente la API.
Implementación de HEATEOAS en controladores base	14 de julio de 2026	- Convertir respuestas individuales a <code>EntityModel<T></code>. - Convertir listas a

Imagen de las actividades definidas con un cronograma dentro de la herramienta Notion



Ahora con motivo de que se pueda apreciar mejor la forma en la cual se trabajó se dejará una copia de la tabla de actividades. Esta a diferencia de la de Notion se agregará una columna con el nombre de los integrantes, es debido a que Notion no los muestra a simple vista, pero si uno abre la herramienta se ve el nombre de cada persona responsable.

Actividades	Responsable	Fecha	Detalle de actividades
Levantamiento y Planificación	Raúl Zúñiga y Francisco Henriquez	11/07/2026	- Revisar los requisitos de la actividad. - Identificar entidades, controladores y servicios existentes. - Definir qué partes se documentarán con OpenAPI y cuáles se reforzarán con HATEOAS. - Distribuir responsabilidades entre las dos personas.

Revisión de la estructura del proyecto	Francisco Henríquez	12/07/2026	• Analizar la arquitectura del backend. • Verificar las dependencias en pom.xml. • Revisar la configuración de seguridad, Swagger y las rutas protegidas. • Identificar controladores que aún devolvían respuestas simples.
Implementación de OpenAPI	Raul Zúñiga	13/07/2026	• Agregar o completar las anotaciones @Tag, @Operation y @ApiResponse. • Documentar los endpoints principales. • Verificar que Swagger UI muestre correctamente la API.
Implementación de HEATEOAS en controladores base	Francisco Henríquez	14/07/2026	• Convertir respuestas individuales a EntityModel<T>. • Convertir listas a CollectionModel<EntityModel<T>>. • Agregar enlaces self y enlaces a la colección. • Probar primero en uno o dos controladores como referencia.
Ajuste del resto de controladores	Raúl Zúñiga	15/07/2026	• Replicar el patrón HATEOAS en los controladores restantes. • Homogeneizar las respuestas entre módulos. • Revisar autenticación y login para mantener la consistencia. • Corregir imports, tipos de retorno y enlaces.
Pruebas y validaciones	Francisco Henríquez	16/07/2026	• Ejecutar las pruebas del proyecto. • Probar endpoints en Swagger o Postman. • Corregir errores de compilación o de respuesta. • Verificar que la seguridad y la documentación no se rompan.

Documentación y entrega	Raúl Zúñiga	17/07/2026	• Redactar el informe o el archivo .md. • Resumir qué se hizo, por qué y qué beneficios aporta. • Recopilar evidencias de Swagger, respuestas y pruebas. • Revisar el formato final antes de entregar.
-------------------------	-------------	------------	---

Configuración del frameworks de seguridad

Como objetivo de este punto, se puede indicar que la seguridad del proyecto se configuró para proteger los endpoints del backend usando autenticación basado en JWT y autorización por roles.

El objetivo principal fue:

- a- Permitir el acceso público solo a recursos necesario, como el login y la documentación de Swagger o Postman.
- b- Exigir autenticación para el resto de los endpoints
- c- Restringir acciones sensibles según el rol del usuario.

Componentes que se implementaron en el proyecto para seguridad son:

1- SecurityConfig:

Esto se utilizó para centralizar la configuración de Spring Security:

- Desactiva CSRF, porque la API trabaja como backend REST
- Habilita protección por autenticación para todos los endpoints.
- Permite acceso libre a:
 - `/api/auth/**`
 - `/public/**`
 - `/swagger-ui/**`
 - `/swagger-ui.html`
 - `/v3/api-docs/**`
- Inserta el filtro de JWT antes del filtro de autenticación por usuario y contraseña.
- Desactiva formLogin y httpBasic, porque el acceso se maneja por token

2- JwtAuthenticationFilter

Este filtro revisa cada solicitud HTTP:

- Busca el encabezado Authorization
- Valida que el token venga con el formato Bearer <token>
- Verifica el token con JwtUtil
- Si el token es válido, carga el usuario con CustomUserDetailsService.

-Crea la autenticación en el SecurityContext.

Con esto, spring Security reconoce al usuario autenticado durante la petición.

3- JwtUtil

Esta clase se encarga de:

- Generar tokens JWT
- Incluir el nombre del usuario como subject
- Incluir los roles del usuario dentro del token
- Validar la firma y expiración del token
- Extraer el username desde el token

La configuración usa una clave secreta definida por propiedad:

- minimarket.jwt.secret
- minimarket.jwt.expiration-ms

Si no se define, se usan valores por defecto.

4- AuthController

Este controlador expone el endpoint de autenticación.

- POST /api/auth/login

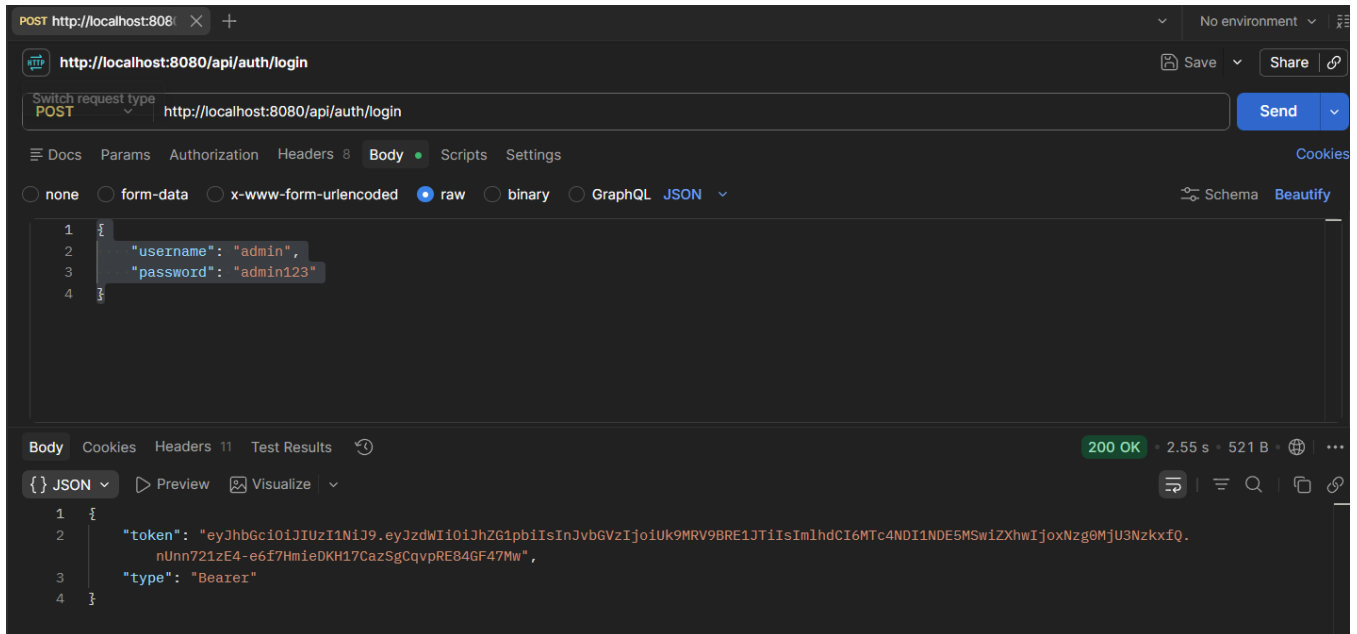
Recibe un JSON:

```
{  
  "username": "admin",  
  "password": "admin123"  
}
```

Si las credenciales son correctas, me va a devolver un 200OK:

- autentica el usuario con AuthenticationManager
- genera un JWT con JwtUtil

- Responde con el token



5- LoginRequest y JwtResponse

Estos modelos representan:

- LoginRequest: datos de entrega para iniciar sesión.
- JwtResponse: respuesta que entrega el token al cliente.

Aurtorización por roles

El proyecto de esta semana se configuró con la autorización a niveles con métodos de `@PreAuthorize`.

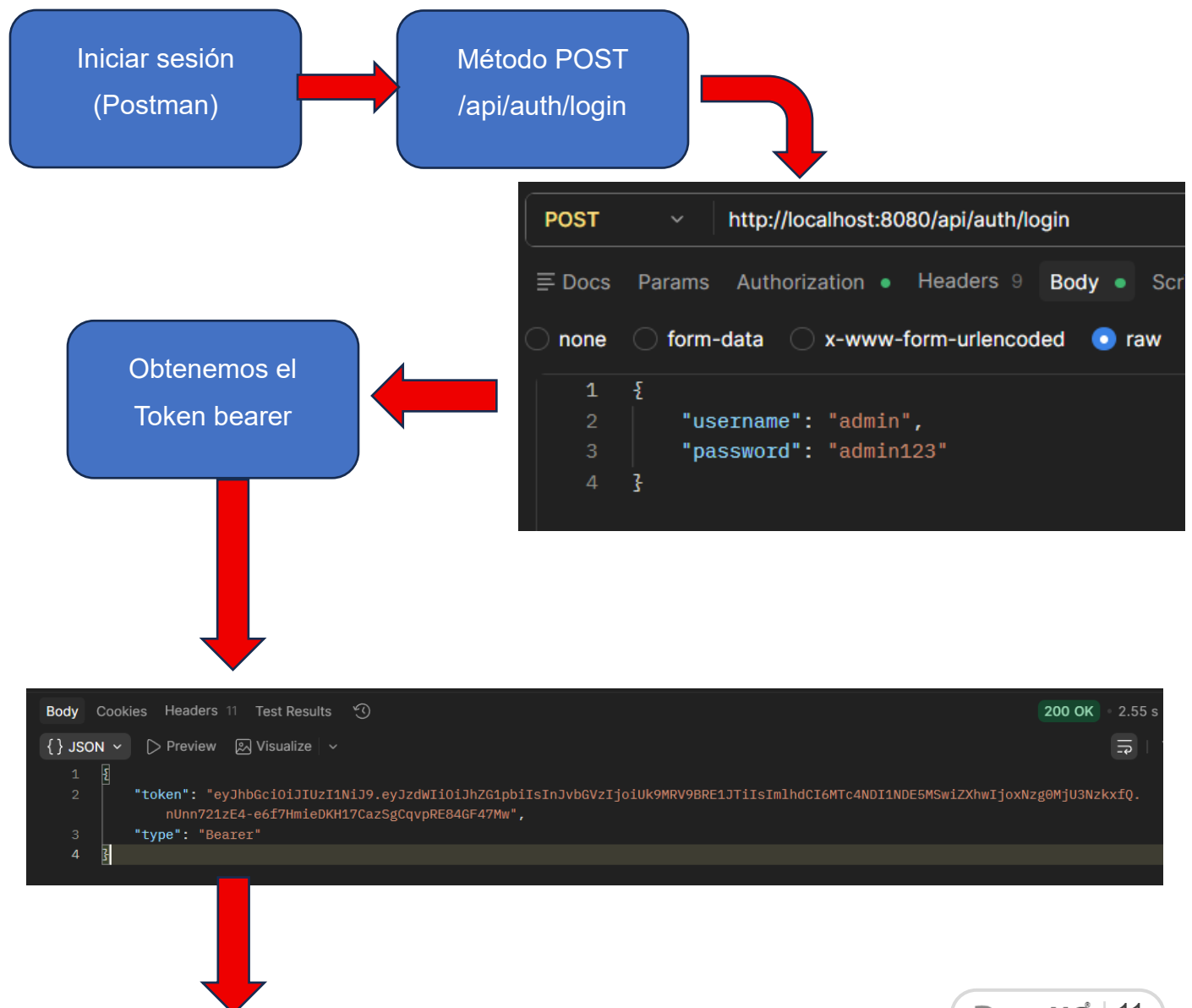
Roles de autorización:

- ROLE_ADMIN
- ROLE_CLIENTE

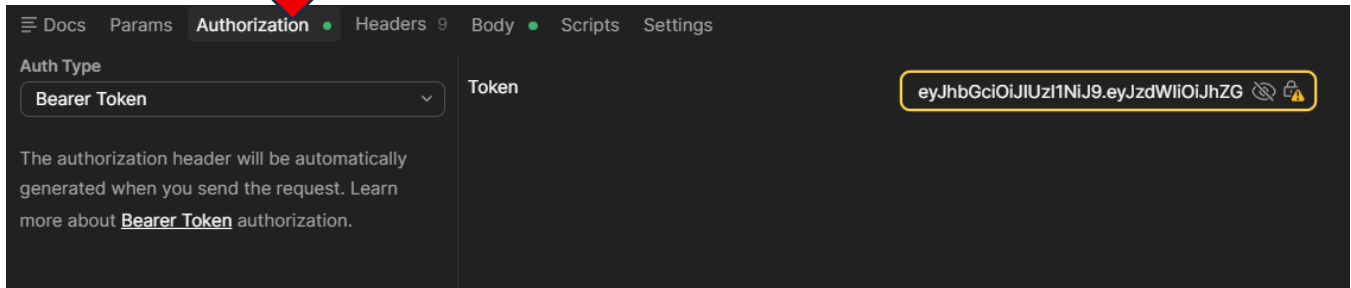
Ejemplo de la proteccion aplicada en el proyecto

	ROL	Escritura	Creación	Eliminación	Edición	Lectura
productoController	ADMIN	☒	☒	☒	☒	☒
	CLIENTE					☒
InventarioController	ADMIN	☒				☒
	CLIENTE					☒
UsuarioController	ADMIN	☒	☒	☒	☒	☒
VentaController	ADMIN		☒			☒
	CLIENTE		☒			
CarritoController	ADMIN					☒
	CLIENTE					☒

Flujo de uso del backend



Authorization (Postman)

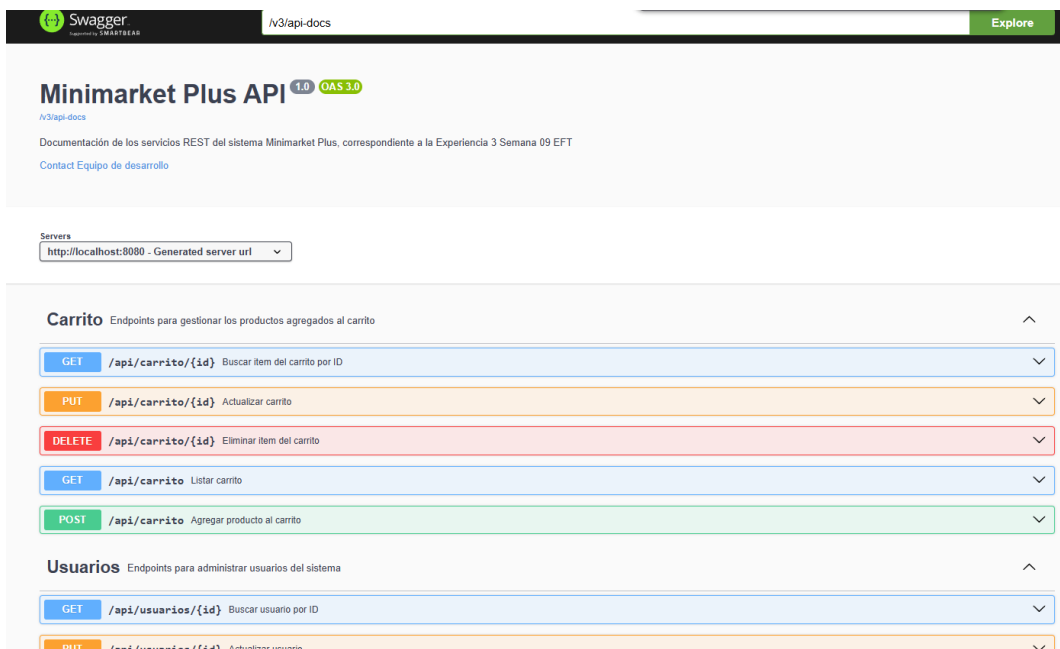


Lo anterior permite el acceso a la aplicación de acuerdo a los permisos que tenga el endpoint. Lo anterior es conforme al rol de usuario que estemos usando.

- Usuario admin entran con el rol `ROLE_ADMIN`
- Usuario cliente entran con el rol `ROLE_CLIENTE`

Esto permite probar rápidamente la autenticación y los permisos, cuando estamos trabajando en POSTMAN, sin embargo, cuando trabajamos en Swagger éste permanece accesible sin autenticación para facilitar las pruebas y documentación:

- `/swagger-ui.html`
- `/v3/api-docs`



Hay que considerar lo siguiente:

- a- La API está pensada para ser utilizada como backend REST, por eso no usa login con formulario, ya que eso es para la rama de los frontend
- b- JWT permite mantener la sesión de forma stateless
- c- Los roles controlan que acciones pueden realizar cada usuario.
- d- La clave secreta JWT debe manejarse con cuidado en entorno productivos.

Detalle de las pruebas unitarias

El proposito de las pruebas unitarias para el proyecto fueron las siguientes:

- Validar el comportamiento de los controladores REST.
- Verificar el comportamiento de los servicios unitaris.
- Probar la lógica de autenticación y seguridad JWT.
- Generar un reporte de cobertura para identificar código no ejecutado.

Herramienta de pruebas unitarias utilizadas	
JUnit 5	Pruebas unitarias.
MocMvc	Para probar endpoints REST.
Mockito	Para simular servicios y repositorios.
Spring Security Test	Para pruebas con usuarios simulados.
JaCoCo	Para medir cobertura de instrucciones, ramas, lineas, métodos y complejidad.

Tipos de pruebas aplicadas

1- Pruebas de Controladores

Se realizaron pruebas con MocMvc sobre los endpoint principales del sistema

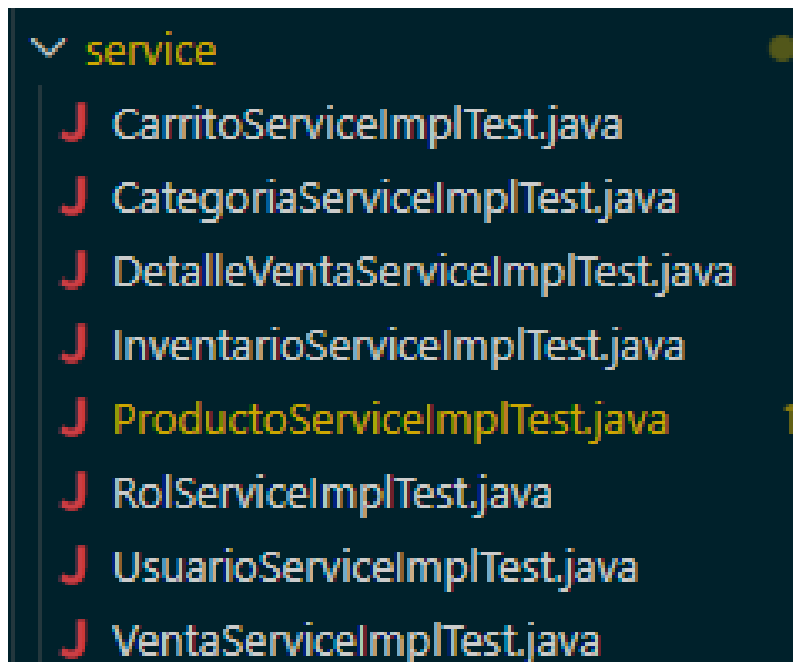
test \ java \ com \ minimarket	
controller	
J CarritoControllerTest.java	1
J CategoriaControllerTest.java	1
J DetalleVentaControllerTest.java	1
J HolaMundoControllerTest.java	
J InventarioControllerTest.java	1
J ProductoControllerTest.java	1
J UsuarioControllerTest.java	1
J VentaControllerTest.java	3

¿Que van a validar estas pruebas?

- Respuestas 200 Ok en casos exitosos
- Respuestas 201/204/404 según el flujo lógico del endpoints
- Operaciones de listado, búsqueda, creación, actualización y eliminación.
- Casos donde el recurso no existe.
- Integración con HEATEOAS en los controladores que lo usan.

2- Pruebas de servicios

Se agregaron pruebas unitarias para validar la lógica de negocio de cada servicio usando Mockito.

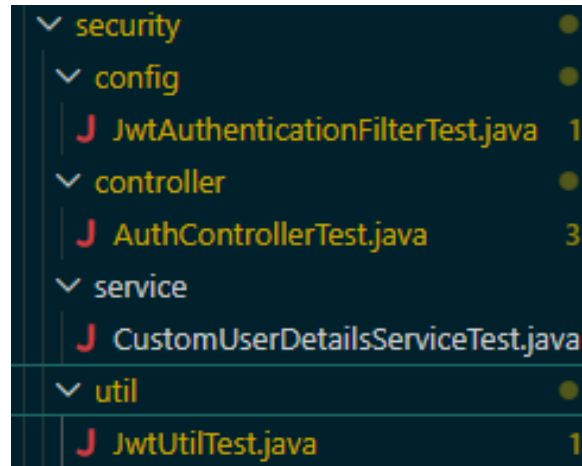


¿Que van a validar estas pruebas?

- Delegación correcta al repositorio
- Retorno de listas.
- Búsqueda por ID.
- Búsqueda por username.
- Búsqueda por relaciones, como findByCategoriald, findByUsuariold y findByVentald
- Eliminación por ID

3- Pruebas de seguridad

Se agregaron pruebas específicas para el flujo JWT



Los componentes que se probaron en estas pruebas fueron:

- JwtUtil
- JwtAuthenticationFilter
- CustomUserDetailsService
- AuthController

¿Que van a validar estas pruebas?

- Generación de token JWT.
- Validación de token correcto.
- Rechazo de token inválido.
- Extracción de username desde el token.
- Autenticación del usuario en el filtro.
- Carga de usuario por username.
- Respuesta correcta del endpoints de login.

CONSIDERACIONES:

Para estas pruebas, se utilizó una estrategia para evitar el código 403 en las pruebas de los controladores, para lo cual se utilizó el `@AutoConfigureMockMvc(addFilters = false)`. Esto desactivó los filtros de seguridad durante el test de controlador, lo que permitió validar la lógica del endpoints sin que Spring Security bloquee la ejecución con 403 Forbidden.

Cuando la seguridad si era parte del objetivo del test, se usó @WithMockUser, apra simular un usuario con el rol necesario.

Para ejecutar la prueba se escribió en la terminal mvn test, lo cual permitió correr las pruebas en el proyecto. En las primeras ejecuciones se generaron 59 pruebas unitarias, de las cuales 0 presentaron fallas, 0 Errores y 0 fueron saltadas y se demoró en ejecutar 27.888 segundos.

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s -- in com.minimarket.service.UsuarioServiceImplTest
[INFO] Running com.minimarket.service.VentaServiceImplTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.114 s -- in com.minimarket.service.VentaServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 59, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ minimarket ---
[INFO] Loading execution data file C:\Users\rzuni\Desktop\Raul\Backend 2\Semana 07 - Documentación de Microservicios con OpenAPI\PB2202_Exp3_S7_Caso_actividad_backend_Minimarket (forma C)\minimarket\target\jacoco.exec
[INFO] Analyzed bundle 'minimarket' with 35 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 27.888 s
[INFO] Finished at: 2026-07-16T22:53:29-04:00
[INFO] -----
PS C:\Users\rzuni\Desktop\Raul\Backend 2\Semana 07 - Documentación de Microservicios con OpenAPI\PB2202_Exp3_S7_Caso_actividad_backend_Minimarket (forma C)\minimarket>
```

Además el resultado que fueron presentados en el reporte de JaCoCo, nos entregó un 75% de cubrimiento para pruebas de instrucciones y un 18% para las pruebas de ramas (desiciones).

 minimarket
  [Session](#)

minimarket

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Clas
com.minimarket.controller		60 %		13 %	34 66	62 140	16 48	0
com.minimarket.security.util		3 %		n/a	6 7	24 25	6 7	0
com.minimarket.security.config		58 %		0 %	5 11	12 31	1 7	0
com.minimarket.security.model		58 %		n/a	8 19	11 29	8 19	0
com.minimarket.entity		92 %		n/a	5 74	8 107	5 74	0
com.minimarket.security.controller		85 %		n/a	0 2	2 9	0 2	0
com.minimarket		37 %		n/a	1 2	2 3	1 2	0
com.minimarket.config		100 %		100 %	0 19	0 98	0 17	0
com.minimarket.service.impl		100 %		n/a	0 42	0 48	0 42	0
com.minimarket.security.service		100 %		n/a	0 3	0 4	0 3	0
Total	447 of 1.848	75 %	39 of 48	18 %	59 245	121 494	37 221	0

Created with [JaCoCo](#) 0.8.12.2024033108

Al observar las pruebas, pudimos indentificar el teníamos una debilidad con las pruebas realizadas en el controller, respecto al cubrimiento de las ramas (13%)

minimarket > com.minimarket.controller

[Source Files](#) [Sessi](#)

com.minimarket.controller

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
InventarioController		56 %		0 %	6	10	14	26	3	7	0	1
DetalleVentaController		19 %		0 %	6	9	12	15	3	6	0	1
CategoriaController		19 %		0 %	6	9	12	15	3	6	0	1
UsuarioController		64 %		25 %	6	11	9	25	4	9	0	1
CarritoController		67 %		16 %	5	10	10	26	2	7	0	1
VentaController		48 %		0 %	2	5	2	5	1	4	0	1
ProductoController		93 %		50 %	3	10	3	26	0	7	0	1
HolaMundoController		100 %		n/a	0	2	0	2	0	2	0	1
Total	257 of 644	60 %	31 of 36	13 %	34	66	62	140	16	48	0	8

Created with JaCoCo 0.8.12.20240331

Si bien es cierto que la pauta de evaluación no nos indica que % que deben cubrir las pruebas, se revisó en clases que una buena practica es cubrir el 80% de las pruebas de instrucciones e intentar de cubrir un 70% para las pruebas de las desiciones basado en ramas. Es por dicha razón que se implementaron más pruebas con el objetivo de cubrir el porcentaje. Aumentando las pruebas a 86 test ejecutados con 0 errores, 0 fallas y 0 saltos.

```
[INFO] Results:
[INFO]
[INFO] Tests run: 86, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ minimarket ---
[INFO] Loading execution data file C:\Users\rzuni\Desktop\Raul\Backend 2\Semana 07 - Documentación de Microservicios con OpenAPI\PBV2202_Exp3_S
7_Caso_actividad_backend_Minimarket (forma C)\minimarket\target\jacoco.exec
[INFO] Analyzed bundle 'minimarket' with 35 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 26.014 s
[INFO] Finished at: 2026-07-16T23:58:37-04:00
[INFO]
```

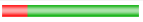
Las pruebas de JaCoCo nos arrojó con esta implementación en un 92% de cubrimiento de pruebas de instrucciones y un 70% de cubrimiento para las pruebas de las ramas.

minimarket

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.minimarket.controller		85 %		63 %	14	66	18	140	1	48	0	8
com.minimarket.entity		92 %		n/a	5	74	8	107	5	74	0	8
com.minimarket.security.model		79 %		n/a	6	19	8	29	6	19	0	3
com.minimarket.security.controller		85 %		n/a	0	2	2	9	0	2	0	1
com.minimarket		37 %		n/a	1	2	2	3	1	2	0	1
com.minimarket.config		100 %		100 %	0	19	0	98	0	17	0	2
com.minimarket.service.impl		100 %		n/a	0	42	0	48	0	42	0	8
com.minimarket.security.config		100 %		87 %	1	11	0	31	0	7	0	2
com.minimarket.security.util		100 %		n/a	0	7	0	25	0	7	0	1
com.minimarket.security.service		100 %		n/a	0	3	0	4	0	3	0	1
Total	140 of 1.848	92 %	14 of 48	70 %	27	245	38	494	13	221	0	35

Ahora en las ramas, de los controladores, se trabajaron aquellos controladores que no tenían pruebas de ramas y se logró cubrir en ellas en un 50%

com.minimarket.controller

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
InventarioController		80 %		50 %	3	10	5	26	0	7	0	1
DetalleVentaController		68 %		50 %	3	9	4	15	0	6	0	1
CategoriaController		68 %		50 %	3	9	4	15	0	6	0	1
UsuarioController		87 %		75 %	2	11	2	25	1	9	0	1
CarritoController		86 %		66 %	2	10	3	26	0	7	0	1
VentaController		88 %		50 %	1	5	0	5	0	4	0	1
ProductoController		100 %		100 %	0	10	0	26	0	7	0	1
HolaMundoController		100 %		n/a	0	2	0	2	0	2	0	1
Total	96 of 644	85 %	13 of 36	63 %	14	66	18	140	1	48	0	8

Con esto deriamos por cumplimiento a las buenas practicas de las pruebas unitarias.

Las pruebas unitarias implementadas permiten validar el funcionamiento principal del bankend y generan un reporte de JaCoCo para evidenciar calidad de pruebas.

Mejoras aplicadas

Las mejoras de esta actividad fue reparar y homogenizar el backend de Minimarket para que los controladores expongan respuestas enriquecidas con HATEOAS, manteniendo la estructura REST existente y sin alterar las rutas del proyecto.

Con esto se buscó:

- Mejorar la navegabilidad entre recursos;
- estandarizar las respuestas de la API;
- Facilitar el consumo desde clientes externos;
- Dejar el backend mejor reparado para futuras integraciones.

Controladores actualizados

Se revisaron y ajustaron los controladores que todavía devolvían entidades simples o listas planas.

Los controladores alineados con HATEOAS fueron:

- DetalleVentasController.java
- CategoriaController.java
- VentaController.java
- AuthController.java

Además, se verificó que InventarioController ya seguía el mismo enfoque HATEOAS, por lo que quedó como referencia del patrón esperado.

Cambios principales aplicados

Respuestas individuales.

Las respuestas de recursos individuales se envolvieron con EntityModel<T>

Esto permite ajustar enlaces útiles dentro del mismo JSON, por ejemplo:

- self
- enlace a la colección general
- enlace relacionados al recurso

Listado de recursos

Las respuesta de colección se cambiaron a `CollectionModel<EntityModel<T>>`

Con esto, caada elemento del listado puede incluir propios enlaces y la colección completa puede exponer también un enlace self

Autenticación

El endpoint de login ahora devuelve el `JWTResponse` dentro de `EntityModel<JwtResponse>`
Esto mantiene consistencia con el resto del proyecto y deja la puerta abierta para añadir enlaces adicionales relacioados con sesión o navegación del API.

Beneficios que se obtubieron

Las mejoras aplicadas aportaron varios beneficios:

- maryo consistencia en todas las respuestas del backend;
- mejor experiencia para consumidores del API;
- navegación más clara entre recursos;
- estrutura más mantenible;
- base más solida para documentación con OpenAPI y pruebas funcionales.

Resultado final de las implementaciones de esta semana

Luego de estos cambios, el proyecto quedó con un estilo más uniforme en sus contraladores principales.

La API conserva sus rutas actuales, pero ahora entrega respuestas más descriptivas y navegables, alineadas con el enfoque HEATEOS que ya usaban los otros controladores que fueron trabajados durante la semana 8.

Documentación técnica de la API utilizando estándares OpenAPI y Specification (OAS) y HEATEOAS

El trabajo previo fue clave para esta semana, porque permitió documentar endpoints reales sobre una estructura de servicios ya establecidos y protegidas.

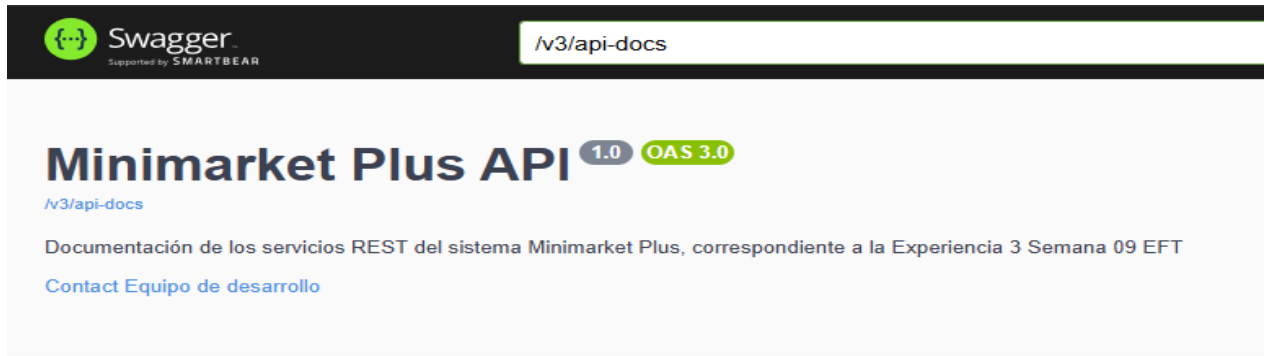
Con esa base, el proyecto avanzó hacia una capa más madura de exposición de servicios:

- Se incorporó *springdoc-openapi* a nuestro archivo pom.xml, para generar documentación automática y una interfaz Swagger UI. Para ello se incorporó la dependencia correspondiente.

```
45      <dependency>
46      |      <groupId>org.springdoc</groupId>
47      |      <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
48      |      <version>2.6.0</version>
49      </dependency>
```

- Se ajustó la configuración de seguridad para permitir el acceso a `/swagger-ui.html` y `/v3/api-docs`

```
J OpenApiConfig.java •
src > main > java > com > minimarket > config > J OpenApiConfig.java > ...
4  import io.swagger.v3.oas.models.info.Contact;
5  import io.swagger.v3.oas.models.info.Info;
6  import org.springframework.context.annotation.Bean;
7  import org.springframework.context.annotation.Configuration;
8
9  @Configuration
10 public class OpenApiConfig {
11
12     @Bean
13     public OpenAPI minimarketOpenAPI() {
14         return new OpenAPI()
15             .info(new Info()
16                 .title(title: "Minimarket Plus API")
17                 .version(version: "1.0")
18                 .description(description: "Documentación de los servicios REST del sistema Minimarket Plus, correspondiente a la API")
19                 .contact(new Contact()
20                     .name(name: "Equipo de desarrollo")
21                     .email(email: "rau.zuniga@duocuc.cl")));
22     }
23 }
24
```



- Se anotaron los controladores principales con metadatos OpenAPI (Carrito, Ventas, Usuarios, Productos, Autenticación, Categorías, Inventario y Detalle de ventas). Los endpoints principales que se diseñaron siguieron una estructura REST coherente:
 - GET: para consulta de recursos
 - POST: para creación
 - PUT: para actualización
 - DELETE: para eliminación

La organización por recursos (/api/productos, /api/ventas, /api/carrito, /api/inventario, /api/usuarios, /api/auth/login, /api/categorías, /api/inventario, /api/detalle-venta) facilita la lectura del API y se alinea con OpenAPI Specification al exponer rutas claras, verbos HTTP consistentes y descripciones formales de respuesta.

Imagen de la clase CarritoController.java

```
@RestController
@RequestMapping("/api/carrito")
@Tag(name = "Carrito", description = "Endpoints para gestionar los productos agregados al carrito")
public class CarritoController {

    @Autowired
    private CarritoService carritoService;

    @GetMapping
    @Operation(summary = "Listar carrito", description = "Obtiene todos los items del carrito")
    public List<Carrito> listarCarrito() {
        return carritoService.findAll();
    }

    @GetMapping("/{id}")
    @Operation(summary = "Buscar item del carrito por ID", description = "Retorna un item específico del carrito")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Item encontrado"),
        @ApiResponse(responseCode = "404", description = "Item no encontrado")
    })
    public ResponseEntity<Carrito> obtenerCarritoPorId(@PathVariable Long id) {
        Carrito carrito = carritoService.findById(id);
        return (carrito != null) ? ResponseEntity.ok(carrito) : ResponseEntity.notFound().build();
    }
}
```

Imagen de DetalleVentaController.java

```
16 @RestController
17 @RequestMapping("/api/detalle-ventas")
18 @Tag(name = "Detalle de ventas", description = "Endpoints para administrar los detalles de cada venta")
19 public class DetalleVentaController {
20
21     @Autowired
22     private DetalleVentaService detalleVentaService;
23
24     @GetMapping
25     @PreAuthorize("hasRole('ADMIN')")
26     @Operation(summary = "Listar detalles de venta", description = "Obtiene todos los detalles de venta registrados")
27     @ApiResponses({
28         @ApiResponse(responseCode = "200", description = "Detalles obtenidos correctamente"),
29         @ApiResponse(responseCode = "401", description = "No autorizado"),
30         @ApiResponse(responseCode = "403", description = "Acceso denegado")
31     })
32     public List<DetalleVenta> listarDetalleVentas() {
33         return detalleVentaService.findAll();
34     }
35
36     @GetMapping("/{id}")
37     @PreAuthorize("hasRole('ADMIN')")
38     @Operation(summary = "Buscar detalle de venta por ID", description = "Retorna un detalle de venta específico según su id")
39     @ApiResponses({
```


Imagen de UsuarioController.java

```
src > main > java > com > minimarket > controller > UsuarioController.java > UsuarioController
17 @RestController
18 @RequestMapping("/api/usuarios")
19 @Tag(name = "Usuarios", description = "Endpoints para administrar usuarios del sistema")
20 public class UsuarioController {
21
22     @Autowired
23     private UsuarioService usuarioService;
24
25     @GetMapping
26     @Operation(summary = "Listar usuarios", description = "Obtiene todos los usuarios registrados en el sistema")
27     @ApiResponses({
28         @ApiResponse(responseCode = "200", description = "Lista de usuarios obtenida correctamente"),
29         @ApiResponse(responseCode = "401", description = "No autorizado")
30     })
31     public List<Usuario> listarUsuarios() {
32         return usuarioService.findAll();
33     }
34
35     @GetMapping("/{id}")
36     @Operation(summary = "Buscar usuario por ID", description = "Retorna un usuario especifico segun su identificador")
37     @ApiResponses({
38         @ApiResponse(responseCode = "200", description = "Usuario encontrado"),
39         @ApiResponse(responseCode = "404", description = "Usuario no encontrado"),
40         @ApiResponse(responseCode = "401", description = "No autorizado")
41     })
42     public ResponseEntity<Usuario> obtenerUsuarioPorId(@PathVariable Long id) {
```

Imagen de ProductosController.java

```
J OpenApiConfig.java J UsuarioController.java J ProductoController.java X
src > main > java > com > minimarket > controller > ProductoController.java > {} com.minimarket.controller
19
20 import java.util.List;
21
22 @RestController
23 @RequestMapping("/api/productos")
24 @Tag(name = "Productos", description = "Endpoints para administrar productos del minimarket")
25 public class ProductoController {
26
27     @Autowired
28     private ProductoService productoService;
29
30     @GetMapping
31     @Operation(summary = "Listar productos", description = "Obtiene todos los productos registrados")
32     public List<Producto> listarProductos() {
33         return productoService.findAll();
34     }
35
36     @GetMapping("/{id}")
37     @Operation(summary = "Buscar producto por ID", description = "Retorna un producto específico según su identificador")
38     @ApiResponses({
39         @ApiResponse(responseCode = "200", description = "Producto encontrado"),
40         @ApiResponse(responseCode = "404", description = "Producto no encontrado")
41     })
42     public ResponseEntity<Producto> obtenerProductoPorId(@PathVariable Long id) {
43         Producto producto = productoService.findById(id);
44         return (producto != null) ? ResponseEntity.ok(producto) : ResponseEntity.notFound().build();
45     }
46 }
```

Imagen de CategoriasController.java

```
c > main > java > com > minimarket > controller > J CategoriaController.java > CategoriaController
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api/categorias")
14 @Tag(name = "Categorías", description = "Endpoints para administrar categorías de productos")
15 public class CategoriaController {
16
17     @Autowired
18     private CategoriaService categoriaService;
19
20     @GetMapping
21     public List<Categoria> listarCategorias() {
22         return categoriaService.findAll();
23     }
24
25     @GetMapping("/{id}")
26     public ResponseEntity<Categoria> obtenerCategoriaPorId(@PathVariable Long id) {
27         Categoria categoria = categoriaService.findById(id);
28         return (categoria != null) ? ResponseEntity.ok(categoria) : ResponseEntity.notFound().build();
29     }
30
31     @PostMapping
32     public ResponseEntity<Categoria> crearCategoria(Categoria categoria) {
33         return ResponseEntity.ok(categoriaService.save(categoria));
34     }
35
36     @DeleteMapping("/{id}")
37     public ResponseEntity<Void> eliminarCategoria(@PathVariable Long id) {
38         categoriaService.deleteById(id);
39         return ResponseEntity.noContent().build();
40     }
41
42     @PutMapping("/{id}")
43     public ResponseEntity<Categoria> actualizarCategoria(@PathVariable Long id, Categoria categoria) {
44         categoria.setId(id);
45         return ResponseEntity.ok(categoriaService.save(categoria));
46     }
47 }
```

InventarioController.java

```
21 @RestController
22 @RequestMapping("/api/inventario")
23 @Tag(name = "Inventario", description = "Endpoints para registrar y consultar movimientos de inventario")
24 public class InventarioController {
25
26     @Autowired
27     private InventarioService inventarioService;
28
29     @GetMapping
30     @PreAuthorize("hasAnyRole('ADMIN','CLIENTE')")
31     @Operation(summary = "Listar inventario", description = "Obtiene todos los movimientos de inventario")
32     @ApiResponses({
33         @ApiResponse(responseCode = "200", description = "Movimientos obtenidos correctamente"),
34         @ApiResponse(responseCode = "401", description = "No autorizado")
35     })
36     public CollectionModel<EntityModel<Inventario>> listarMovimientosDeInventario() {
37         List<EntityModel<Inventario>> movimientos = inventarioService.findAll().stream()
38             .map(this::toModel)
39             .toList();
40         return CollectionModel.of(movimientos,
41             linkTo(methodOn(InventarioController.class).listarMovimientosDeInventario()).withSelfRel());
42     }
43 }
```

VentaController.java

```
16 @RestController
17 @RequestMapping("/api/ventas")
18 @Tag(name = "Ventas", description = "Endpoints para consultar y registrar ventas del minimarket")
19 public class VentaController {
20
21     @Autowired
22     private VentaService ventaService;
23
24     @GetMapping
25     @PreAuthorize("hasRole('ADMIN')")
26     @Operation(summary = "Listar ventas", description = "Obtiene el listado completo de ventas registradas")
27     @ApiResponses({
28         @ApiResponse(responseCode = "200", description = "Ventas obtenidas correctamente"),
29         @ApiResponse(responseCode = "401", description = "No autorizado"),
30         @ApiResponse(responseCode = "403", description = "Acceso denegado")
31     })
32     public List<Venta> listarVentas() {
33         return ventaService.findAll();
34     }
35
36     @GetMapping("/{id}")
```

Imagen de la representación de las anotaciones en OpenAPI usando Swagger

Swagger
Supported by SMARTBEAR

/v3/api-docs [Explore](#)

Minimarket Plus API ^{1.0} OAS 3.0

[/v3/api-docs](#)

Documentación de los servicios REST del sistema Minimarket Plus, correspondiente a la Experiencia 3 Semana 09 EFT

[Contact Equipo de desarrollo](#)

Servers
http://localhost:8080 - Generated server url

- Carrito** Endpoints para gestionar los productos agregados al carrito
- Ventas** Endpoints para consultar y registrar ventas del minimarket
- Usuarios** Endpoints para administrar usuarios del sistema
- Productos** Endpoints para administrar productos del minimarket
- Autenticación** Endpoints para iniciar sesión y obtener token JWT
- Categorías** Endpoints para administrar categorías de productos
- Inventario** Endpoints para registrar y consultar movimientos de inventario
- Detalle de ventas** Endpoints para administrar los detalles de cada venta

- Se preparó el escenario para el uso de HATEOAS, con el objetivo de mejorar la navegabilidad entre los recursos.

Se inició la implementación de HATEOAS agregando la dependencia a nuestro archivo POM.XML

```
<dependency>
| <groupId>org.springframework.boot</groupId>
| <artifactId>spring-boot-starter-hateoas</artifactId>
|</dependency>
```

Y se va a usar un enfoque para que este proyecto envuelva respuestas individuales en con *EntityModel* y/o colecciones con *CollectionModel* Esto nos va a permitir agregar enlaces como *self*, *collection*, *update* y *delete*.

Ejemplo:

```
private EntityModel<Producto> toModel(Producto producto) {
    EntityModel<Producto> model = EntityModel.of(producto);
    model.add(linkTo(methodOn(ProductoController.class).obtenerProductoPorId(producto.getId())).withSelfRel());
    model.add(linkTo(methodOn(ProductoController.class).listarProductos()).withRel("productos"));
    return model;
}
```

En términos de calidad técnica, este avance transforma el backend desde una API funcional hacia una API mejor documentada, más fácil de consumir y más clara para integraciones externas.

1. Configuración del entorno de desarrollo

- Detalla los pasos clave para configurar las dependencias necesarias en el proyecto (OpenAPI, HATEOAS).
- Anexa una captura de la estructura de carpetas y del archivo `pom.xml` o equivalente, mostrando las dependencias utilizadas.

Los pasos claves para configurar las dependencias necesarias para el proyecto, fueron la implementación en nuestro archivo POM.XML de las dependencias necesarias para que se pueda ejecutar estos archivos.

Implementación de la dependencia OPENAPI	<pre> 45 <dependency> 46 <groupId>org.springdoc</groupId> 47 <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId> 48 <version>2.6.0</version> 49 </dependency> </pre>
Implementación de la dependencia HATEOAS	<pre> <dependency> <groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-hateoas</artifactId> </dependency> </pre>

Se definió una configuración global de OpenAPI con título, versión, descripción y contacto, Además se utilizaron las anotaciones `@Tag`(Para agrupar funcionalmente), `@Operation`(Para describir la intención de cada método) y `@ApiResponses`(Para declarar respuestas esperadas y errores) en los controladores de Productos, Carritos, Usuarios, Categorías e Inventario. Durante la implementación de estos, se encontraron algunas dificultades que es necesarias indicarlas.

- A- La primera dificultad que se nos presentó al momento de realizar este paso fue ver que la dependencia de Springdoc no estaba implementada, lo cual impidió que inicialmente pudiéramos visualizar la documentación en Swagger UI y exportar el JSON a OpenAPI.
- B- La segunda dificultad que es importante documentar, se presentó en la seguridad, ya que el backend protege por defecto casi todos los recursos. Para resolverlo, se permitieron explícitamente las rutas de documentación:


```

/swagger-ui/**
/swagger-ui.html
/v3/api-docs/**

```
- C- También se observó que las respuestas de algunos controladores no eran homogéneas, ya que ciertos métodos devolvían `List<T>`, mientras otros usan `ResponseEntity<T>`. Eso rompía la API, pero si nos obligó a documentar con más cuidado los códigos de respuesta.

Respecto a HATEOAS se preparó el proyecto para incorporar HATEOAS en las respuestas JSON, iniciando con la implementación de las dependencias (antes descritas), esto permitió que se pudieran devolver `@EntityModel` y colecciones con `@CollectionModel`

Implementación de UsuarioController.java

```
private EntityModel<Usuario> toModel(Usuario usuario) {  
    EntityModel<Usuario> model = EntityModel.of(usuario);  
    model.add(linkTo(methodOn(UsuarioController.class).obtenerUsuarioPorId(usuario.getId())).withSelfRel());  
    model.add(linkTo(methodOn(UsuarioController.class).listarUsuarios()).withRel("usuarios"));  
    return model;  
}
```

Implementación de CarritoController.java

```
private EntityModel<Carrito> toModel(Carrito carrito) {  
    EntityModel<Carrito> model = EntityModel.of(carrito);  
    model.add(linkTo(methodOn(CarritoController.class).obtenerCarritoPorId(carrito.getId())).withSelfRel());  
    model.add(linkTo(methodOn(CarritoController.class).listarCarrito()).withRel("carrito"));  
    return model;  
}
```

Implementación de ProductoController.java

```
private EntityModel<Producto> toModel(Producto producto) {  
    EntityModel<Producto> model = EntityModel.of(producto);  
    model.add(linkTo(methodOn(ProductoController.class).obtenerProductoPorId(producto.getId())).withSelfRel());  
    model.add(linkTo(methodOn(ProductoController.class).listarProductos()).withRel("productos"));  
    return model;  
}
```

Implementación de CategoriaController.java

```
private EntityModel<Categoria> toModel(Categoria categoria) {  
    EntityModel<Categoria> model = EntityModel.of(categoria);  
    model.add(linkTo(methodOn(controller: CategoriaController.class).obtenerCategoriaPorId(categoria.getId())).withSelfRel());  
    model.add(linkTo(methodOn(controller: CategoriaController.class).listarCategorias()).withRel(rel: "categorias"));  
    return model;  
}
```

Implementación de DetalleVentaController.java

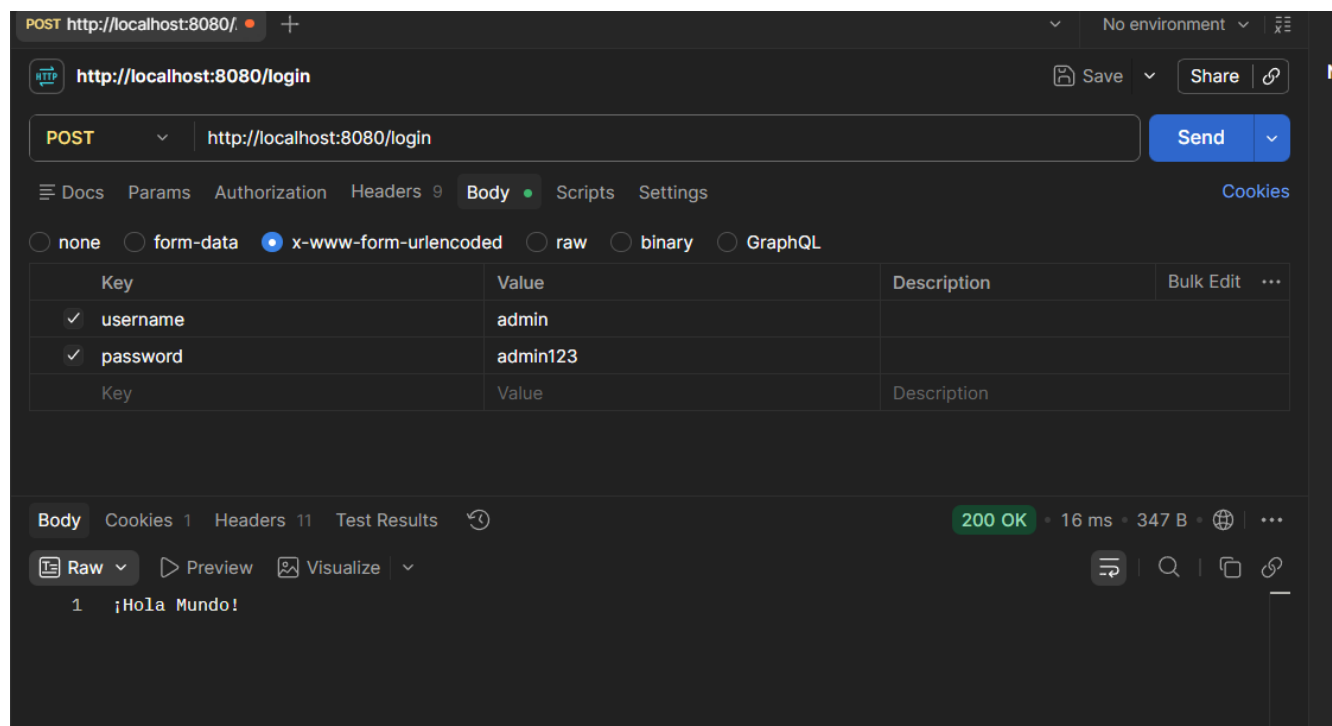
```
private EntityModel<DetalleVenta> toModel(DetalleVenta detalleVenta) {  
    EntityModel<DetalleVenta> model = EntityModel.of(detalleVenta);  
    model.add(linkTo(methodOn(controller: DetalleVentaController.class).obtenerDetalleVentaPorId(detalleVenta.getId())).withSelfRel());  
    model.add(linkTo(methodOn(controller: DetalleVentaController.class).listarDetalleVentas()).withRel(rel: "detalle-venta"));  
    return model;  
}
```

2. Implementación de HATEOAS

- Muestra ejemplos de respuestas JSON con enlaces dinámicos generados por HATEOAS en los endpoints principales.
- Incluye una captura o fragmento de código que demuestre cómo se añadieron los enlaces utilizando `EntityModel`.

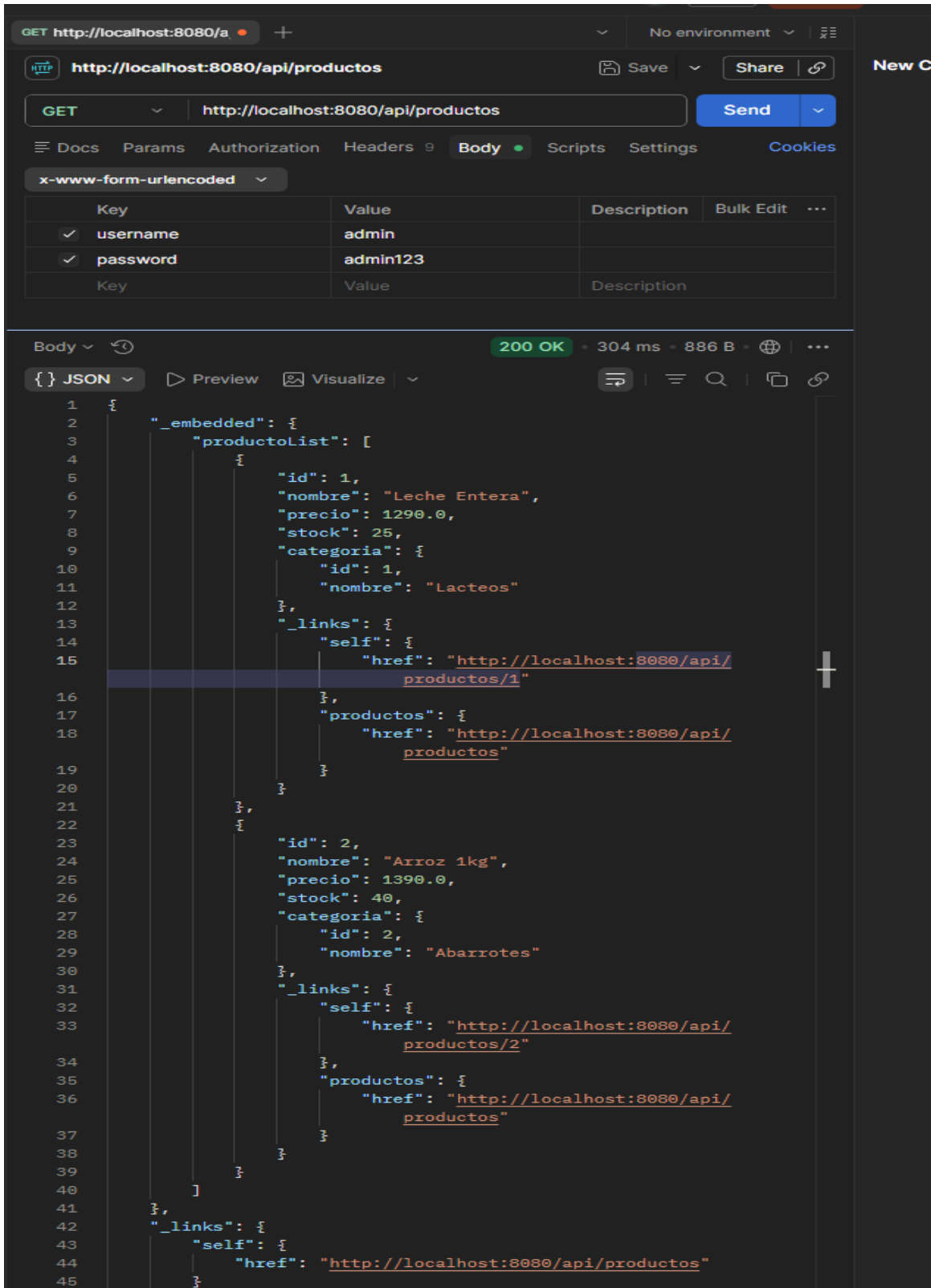
Las pruebas de las respuestas JSON y los enlaces dinámicos de HEATEOAS, fueron realizadas utilizando la herramienta POSTMAN, esto se debió a que como la aplicación requiere de el ingreso de un usuario y contraseña, no supe como agregar esos campos en SWAGGER razón por la cual las capturas serán de esta herramienta:

En POSTMAN con la aplicación ya ejecutándose, lo primero que realizamos fue seleccionar el método POST, posteriormente se agregó la siguiente ruta <http://localhost:8080/login>, después nos fuimos a Body y seleccionamos x-www-form-urlencoded y agregamos en la columna Key username y password y donde dice Value se agregó admin y admin123, finalizamos dándole clic al botón Send y nos devolvió una respuesta 200 Ok y un Hola mundo en consola, lo cual nos permite saber que se ha logeado de forma correcta nuestra aplicación.



PRUEBA DE LOS ENDPOINTS

Productos GET /api/productos



GET http://localhost:8080/a

http://localhost:8080/api/productos

GET http://localhost:8080/api/productos

Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

x-www-form-urlencoded

Key	Value	Description	Bulk Edit	...
✓ username	admin			
✓ password	admin123			
Key	Value	Description		

Body

200 OK • 304 ms • 886 B •

JSON Preview Visualize

```
1 {
2   "_embedded": {
3     "productoList": [
4       {
5         "id": 1,
6         "nombre": "Leche Entera",
7         "precio": 1290.0,
8         "stock": 25,
9         "categoria": {
10          "id": 1,
11          "nombre": "Lacteos"
12        },
13        "_links": {
14          "self": {
15            "href": "http://localhost:8080/api/productos/1"
16          },
17          "productos": {
18            "href": "http://localhost:8080/api/productos"
19          }
20        }
21      },
22      {
23        "id": 2,
24        "nombre": "Arroz 1kg",
25        "precio": 1390.0,
26        "stock": 40,
27        "categoria": {
28          "id": 2,
29          "nombre": "Abarrotes"
30        },
31        "_links": {
32          "self": {
33            "href": "http://localhost:8080/api/productos/2"
34          },
35          "productos": {
36            "href": "http://localhost:8080/api/productos"
37          }
38        }
39      }
40    ]
41  },
42  "_links": {
43    "self": {
44      "href": "http://localhost:8080/api/productos"
45    }
46  }
47 }
```


GET /api/productos Listar productos

Obtiene todos los productos registrados

Parameters
Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/productos' \
  -H 'accept: */*'

```

Request URL

```
http://localhost:8080/api/productos

```

Server response

Code Details

200

Response body

```
{
  "_embedded": {
    "productList": [
      {
        "id": 1,
        "nombre": "Leche Entera",
        "precio": 1200,
        "stock": 25,
        "categoria": {
          "id": 1,
          "nombre": "Lacteos"
        },
        "_links": {
          "self": {
            "href": "http://localhost:8080/api/productos/1"
          },
          "productos": {
            "href": "http://localhost:8080/api/productos"
          }
        }
      },
      {
        "id": 2,
        "nombre": "Arroz 1kg",
        "precio": 1300,
        "stock": 40,
        "categoria": {
          "id": 2,
          "nombre": "Abarrotes"
        }
      }
    ]
  }
}

```

Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/hal+json
date: Sun, 12 Jul 2026 15:34:03 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0

```

Productos GET /api/productos/{id}

GET http://localhost:8080/a

http://localhost:8080/api/productos/1

GET http://localhost:8080/api/productos/1

Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

x-www-form-urlencoded

Key	Value	Description	Bulk Edit	...
☒ username	admin			
☒ password	admin123			
Key	Value	Description		

Body 200 OK • 587 ms • 563 B

{ } JSON Preview Visualize

```

1  {
2    "id": 1,
3    "nombre": "Leche Entera",
4    "precio": 1290.0,
5    "stock": 25,
6    "categoria": {
7      "id": 1,
8      "nombre": "Lacteos"
9    },
10   "_links": {
11     "self": {
12       "href": "http://localhost:8080/api/productos/1"
13     },
14     "productos": {
15       "href": "http://localhost:8080/api/productos"
16     }
17   }
18 }
```

Productos Endpoints para administrar productos del minimarket

GET /api/productos/{id} Buscar producto por ID

Retorna un producto específico según su identificador

Parameters

Cancel

Name Description

id * required

integer(\$int64)
(path)

1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/productos/1' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/productos/1

Server response

Code

Details

200

Response body

```
{
  "id": 1,
  "nombre": "Leche Entera",
  "precio": 1290,
  "stock": 25,
  "categoria": {
    "id": 1,
    "nombre": "Lacteos"
  },
  "_links": {
    "self": {
      "href": "http://localhost:8080/api/productos/1"
    },
    "productos": {
      "href": "http://localhost:8080/api/productos"
    }
  }
}
```



Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/hal+json
date: Sun, 12 Jul 2026 15:30:31 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Responses

Carrito GET /api/carrito

http://localhost:8080/api/carrito

GET http://localhost:8080/api/carrito

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

Key	Value	Description	Bulk Edit
✓ username	admin		
✓ password	admin123		
Key	Value	Description	

Body 200 OK · 44 ms · 1.22 KB

JSON Preview Visualize

```

1 {
2   "_embedded": {
3     "carritolist": [
4       {
5         "id": 1,
6         "usuario": {
7           "id": 2,
8           "username": "cliente",
9           "password": "$2a$10$zxCpmvzG@GWe2hVhkjS9Ke4k58Gs7Ph7cvJChNP.r31b.0o31TKp2",
10          "roles": [
11            {
12              "id": 2,
13              "nombre": "ROLE_CLIENTE"
14            }
15          ]
16        },
17        "producto": {
18          "id": 1,
19          "nombre": "Leche Entera",
20          "precio": 1290.0,
21          "stock": 25,
22          "categoria": {
23            "id": 1,
24            "nombre": "Lacteos"
25          }
26        },
27        "cantidad": 2,
28        "_links": {
29          "self": {
30            "href": "http://localhost:8080/api/carrito/1"
31          },
32          "carrito": {
33            "href": "http://localhost:8080/api/carrito"
34          }
35        }
36      },
37      {
38        "id": 2,
39        "usuario": {
40          "id": 2,
41          "username": "cliente",
42          "password": "$2a$10$zxCpmvzG@GWe2hVhkjS9Ke4k58Gs7Ph7cvJChNP.r31b.0o31TKp2",
43          "roles": [
44            {
45              "id": 2,
46              "nombre": "ROLE_CLIENTE"
47            }
48          ]
49        },
50        "producto": {
51          "id": 2,
52          "nombre": "Arroz 1kg",
53          "precio": 1390.0,
54          "stock": 40,
55          "categoria": {
56            "id": 2,
57            "nombre": "Abarrotes"
58          }
59        },
60        "cantidad": 1,
61        "_links": {
62          "self": {
63            "href": "http://localhost:8080/api/carrito/2"
64          },
65          "carrito": {
66            "href": "http://localhost:8080/api/carrito"
67          }
68        }
69      }
70    ]
71  },
72  "_links": {
73    "self": {
74      "href": "http://localhost:8080/api/carrito"
75    }
76  }
77 }
```

Carrito Endpoints para gestionar los productos agregados al carrito

GET /api/carrito/{id} Buscar item del carrito por ID

PUT /api/carrito/{id} Actualizar carrito

DELETE /api/carrito/{id} Eliminar item del carrito

GET /api/carrito Listar carrito

Obtiene todos los items del carrito

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/carrito' \
  -H 'accept: */*'

```

Request URL

http://localhost:8080/api/carrito

Server response

Code Details

200

Response body

```
{
  "producto": {
    "id": 2,
    "nombre": "Arroz 1kg",
    "precio": 1390,
    "stock": 40,
    "categoria": {
      "id": 2,
      "nombre": "Abarrotes"
    }
  },
  "cantidad": 1,
  "links": {
    "self": {
      "href": "http://localhost:8080/api/carrito/2"
    }
  },
  "carrito": {
    "href": "http://localhost:8080/api/carrito"
  }
},
{
  "links": {
    "self": {
      "href": "http://localhost:8080/api/carrito"
    }
  }
}

```

Download

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/hal+json
date: Sun, 12 Jul 2026 15:37:52 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0

```

Carrito GET /api/carrito/{id}

GET http://localhost:8080/a + No environment

http://localhost:8080/api/carrito/2 Save Share

GET http://localhost:8080/api/carrito/2 Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

Key	Value	Description	Bulk Edit	...
✓ username	admin			
✓ password	admin123			
Key	Value	Description		

Body 200 OK • 56 ms • 746 B •

{ } JSON Preview Visualize

```

1 {
2   "id": 2,
3   "usuario": {
4     "id": 2,
5     "username": "cliente",
6     "password": "$2a$10$zxCpmvzG0gWe2hVhkjS9Ke4k58Gs7Ph7cvJChNP.r31b.0o31TKp2",
7     "roles": [
8       {
9         "id": 2,
10        "nombre": "ROLE_CLIENTE"
11      }
12    ]
13  },
14  "producto": {
15    "id": 2,
16    "nombre": "Arroz 1kg",
17    "precio": 1390.0,
18    "stock": 40,
19    "categoria": {
20      "id": 2,
21      "nombre": "Abarrotes"
22    }
23  },
24  "cantidad": 1,
25  "_links": {
26    "self": {
27      "href": "http://localhost:8080/api/carrito/2"
28    },
29    "carrito": {
30      "href": "http://localhost:8080/api/carrito"
31    }
32  }
33 }
```

Carrito Endpoints para gestionar los productos agregados al carrito

GET /api/carrito/{id} Buscar item del carrito por ID

Retorna un item específico del carrito

Parameters

Cancel

Name Description

id * required

integer(\$int64)

(path)

2

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/carrito/2' \
  -H 'accept: */*'
```

Request URL

http://localhost:8080/api/carrito/2

Server response

Code Details

200

Response body

```
{
  "id": 2,
  "username": "cliente",
  "password": "$2a$10$zxCPmvz60gMe2hVhkj59Ke4k58Gs7Ph7cv3CMNP.r31b.0o31TKp2",
  "roles": [
    {
      "id": 2,
      "nombre": "ROLE_CLIENTE"
    }
  ],
  "producto": {
    "id": 2,
    "nombre": "Arroz 1kg",
    "precio": 1390,
    "stock": 40,
    "categoria": {
      "id": 2,
      "nombre": "Abarrotes"
    }
  },
  "cantidad": 1,
  "links": {
    "self": {
      "href": "http://localhost:8080/api/carrito/2"
    },
    "carrito": {
      "href": "http://localhost:8080/api/carrito"
    }
  }
}
```

Download

Usuario GET /api/usuarios


<http://localhost:8080/api/usuarios>

 Save
  Share

GET

http://localhost:8080/api/usuarios

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

Key	Value	Description	Bulk Edit	...
✓ username	admin			
✓ password	admin123			
Key	Value	Description		

Body

200 OK • 78 ms • 969 B •

JSON

Preview

Visualize

```

1  {
2    "_embedded": {
3      "usuarioList": [
4        {
5          "id": 1,
6          "username": "admin",
7          "password": "$2a$10$mytqAb0rIQs3KdGgg8PZsuXtsS/K0KBaEU5m09dwTVYPuyB2kYaqS",
8          "roles": [
9            {
10             "id": 1,
11             "nombre": "ROLE_ADMIN"
12            }
13          ],
14          "_links": {
15            "self": {
16              "href": "http://localhost:8080/api/usuarios/1"
17            },
18            "usuarios": {
19              "href": "http://localhost:8080/api/usuarios"
20            }
21          }
22        },
23        {
24          "id": 2,
25          "username": "cliente",
26          "password": "$2a$10$zxCPmvzG0gWe2hVhkjS9Ke4k586s7Ph7cvJChNP.r31b.0o31TKp2",
27          "roles": [
28            {
29              "id": 2,
30              "nombre": "ROLE_CLIENTE"
31            }
32          ],
33          "_links": {
34            "self": {
35              "href": "http://localhost:8080/api/usuarios/2"
36            },
37            "usuarios": {
38              "href": "http://localhost:8080/api/usuarios"
39            }
40          }
41        }
42      ]
43    },
44    "_links": {
45      "self": {
46        "href": "http://localhost:8080/api/usuarios"
47      }
48    }
49  }

```


Usuarios Endpoints para administrar usuarios del sistema

GET /api/usuarios/{id} Buscar usuario por ID

PUT /api/usuarios/{id} Actualizar usuario

DELETE /api/usuarios/{id} Eliminar usuario

GET /api/usuarios Listar usuarios

Obtiene todos los usuarios registrados en el sistema

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/usuarios' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/usuarios
```

Server response

Code Details

200

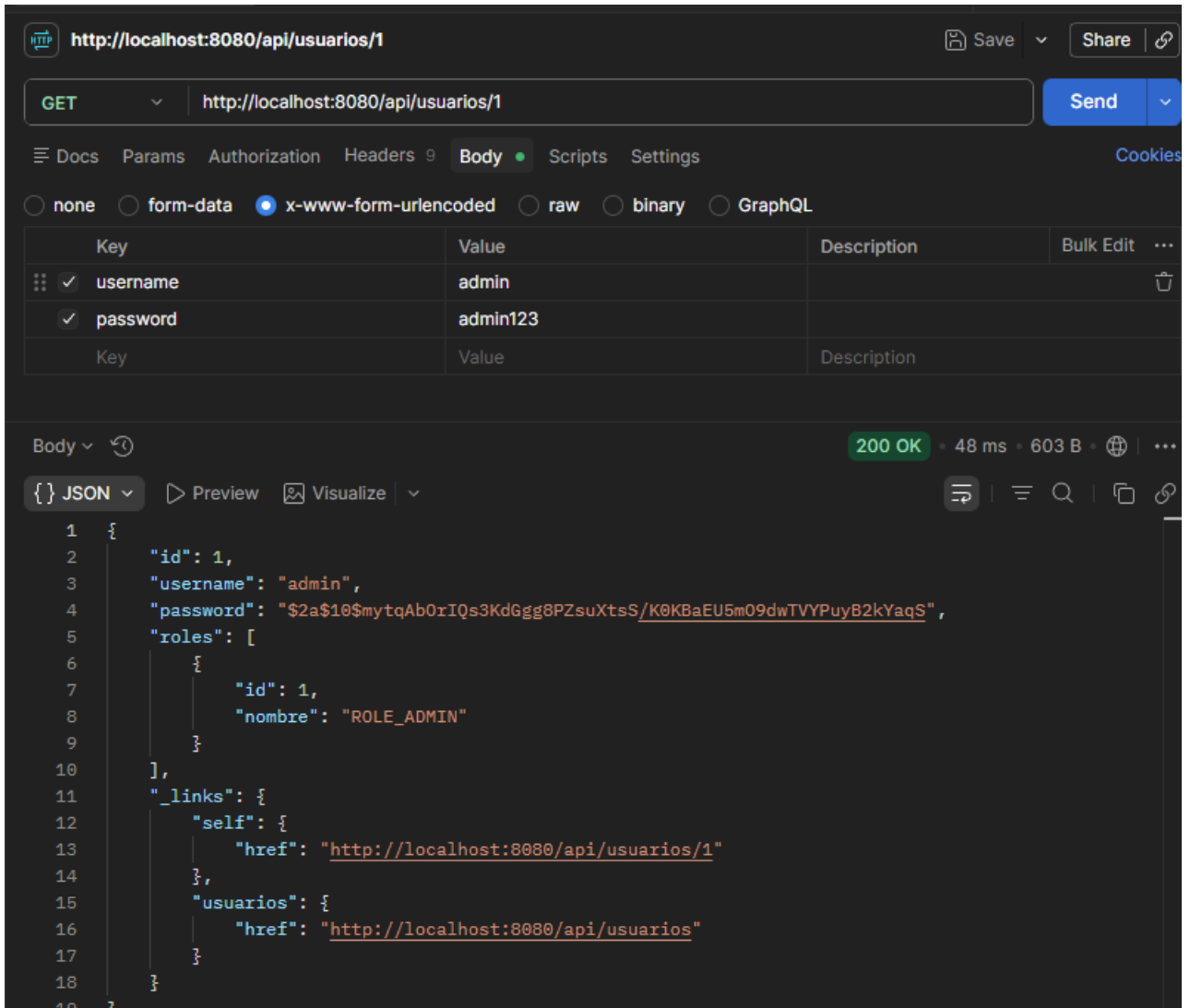
Response body

```
{
  "_embedded": {
    "usuarioList": [
      {
        "id": 1,
        "username": "admin",
        "password": "$2a$10$mytqAbOrIQs3KdGgg8PZsuXts5/K0KBAEU5mO9dwTVYPuy82kYaq5",
        "roles": [
          {
            "id": 1,
            "nombre": "ROLE_ADMIN"
          }
        ],
        "links": {
          "self": {
            "href": "http://localhost:8080/api/usuarios/1"
          },
          "usuarios": {
            "href": "http://localhost:8080/api/usuarios"
          }
        }
      },
      {
        "id": 2,
        "username": "cliente",
        "password": "$2a$10$zxCPmvz60gMe2hVhKjS9Ke4kS8Gs7Ph7cvJChMP.r31b.Oo31TKp2",
        "roles": [
          {
            "id": 2,
            "nombre": "ROLE_CLIENTE"
          }
        ]
      }
    ]
  }
}
```



Download

Usuario GET /api/usuarios/{id}



The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/api/usuarios/1`
- Method:** GET
- Body Type:** x-www-form-urlencoded (selected)
- Form Data:**

Key	Value	Description
username	admin	
password	admin123	
- Response:** 200 OK, 48 ms, 603 B
- Response Body (JSON):**

```
{
  "id": 1,
  "username": "admin",
  "password": "$2a$10$mytqAb0rIQs3KdGgg8PZsuXtsS/K0KBaEU5m09dwTVYPuyB2kYaqS",
  "roles": [
    {
      "id": 1,
      "nombre": "ROLE_ADMIN"
    }
  ],
  "_links": {
    "self": {
      "href": "http://localhost:8080/api/usuarios/1"
    },
    "usuarios": {
      "href": "http://localhost:8080/api/usuarios"
    }
  }
}
```

Usuarios Endpoints para administrar usuarios del sistema

GET `/api/usuarios/{id}` Buscar usuario por ID

Retorna un usuario específico según su identificador

Parameters

Cancel

Name Description

id * required

integer(\$int64)
(path)

1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/usuarios/1' \
  -H 'accept: */*' \
```

Request URL

`http://localhost:8080/api/usuarios/1`

Server response

Code Details

200

Response body

```
{
  "id": 1,
  "username": "admin",
  "password": "$2a$10$mytqAb0rIQs3Kd0gg$PZsuXts/K0K8aEUSm09dwIVVPuy82kYag5",
  "roles": [
    {
      "id": 1,
      "nombre": "ROLE_ADMIN"
    }
  ],
  "links": {
    "self": {
      "href": "http://localhost:8080/api/usuarios/1"
    }
  },
  "usuarios": {
    "href": "http://localhost:8080/api/usuarios"
  }
}
```



Download



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.